Ejercicios tipo examen

4 ejercicios con la estructura del examen de Mongo

Parte	Qué es	Puntos
1. Importar	JSON → MongoDB (`insert_many`)	1
2. Limpiar	Quitar lo malo y convertir texto a números, minúsculas, etc.	3
3. Analizar	Consultas con `aggregate` ($group, $sort, $limit, $match...)	3
4. Presentar	Cada tabla a CSV → Excel → gráficos	3

Cómo usarlos: haz primero el ejercicio tú solo con la plantilla (`plantilla_examen.py`) y los datos de la carpeta. Usa la solución solo para corregirte. **Pista de reparto de tiempo:** la limpieza son 3 puntos y es donde más se falla; dedícale tiempo y comprueba con `print(len(garbiak))` que no se te ha perdido nada raro.

Chuleta de limpieza: «sucio» → «limpio»

Dato sucio	Cómo limpiarlo en Python
`" BILBO "` mayúsculas / espacios	`x.strip().title()` (nombre propio) · `x.strip().lower()` (categoría) · `.capitalize()` (1.ª letra)
`"22 kW"`, `"850 m"`, `"12,5 km"`	`float(x.replace("kW","").replace(",","."))` → quita la unidad y la coma pasa a punto
`"1.250,50 €"` (miles con punto)	`float(x.replace("€","").replace(".","").replace(",","."))` → **primero** quita el punto de miles, **luego** la coma a punto
`"350"` o `"n/a"` o None	`try: int(x) except (ValueError, TypeError): ...` → si falla, descarta o pon None
None o `""` (falta el dato)	`if not d.get("campo"): continue`
Duplicados	`ikusiak = set()` · `if izena in ikusiak: continue` · `ikusiak.add(izena)`
`"Bai"` / `"BAI"` / `"ez"`	`x.strip().lower() == "bai"` → True / False
Fecha `"13/07/2024"`	`data[6:10]` = año · `int(data[3:5])` = mes · `int(data[0:2])` = día
Duración `"3h 30min"`	función `minutuak()` del ejercicio 3: separa por `"h"` y por `"min"`
Lista dentro del documento	limpia cada elemento: `[u.strip().lower() for u in d["lista"]]` · para contar usa `$unwind`
Dato nuevo (derivado)	calcúlalo al limpiar: `round(a / b, 1)` · `año // 10 * 10` (década) · `cantidad * precio`

Qué gráfico poner

Quieres mostrar…	Gráfico
Un valor por categoría (conteos, medias)	Columnas (o barras si los nombres son largos)
Dos valores que se suman por categoría	Columnas apiladas (selecciona solo las columnas de conteo)
Evolución en el tiempo	Líneas (la columna del año/mes no es una serie: es el eje)
Reparto de un total	Sectores (tarta)
Posición en un mapa o dos variables numéricas	Dispersión (X, Y)

Ejercicio 1 · Kargagailuak (puntos de carga de coches eléctricos)

Datos de cargadores de vehículos eléctricos de Euskadi. Hay duplicados, precios que faltan y textos con unidades.

Archivo: `kargagailuak.json` · Solución: `ebazpena1.py`

Campo	Ejemplo (tal como viene, sucio)
izena	"Kargagune Bilbo 12"
udalerria	"BILBO", "bilbo ", " Bilbo"
probintzia	"Bizkaia"
mota	"Azkarra", "azkarra", "AZKARRA ", "Motela"
potentzia	"22 kW", "7,4 kW"
prezioa	"0,52 €/kWh", **""** o **null**
24ordu	"Bai", "BAI", "ez"
lat, lon	"43.26011" (texto)

1. Importar **(1 punto)**

- Carga el JSON en MongoDB (colección `gordinak`).

2. Limpiar **(3 puntos)**

- Lee `gordinak` y guarda en `garbiak` solo los documentos buenos, ya limpios:
- Quita los **duplicados** (mismo *izena*, te quedas con el primero) y los que no tienen *prezioa*.
- *udalerria* con formato nombre propio (*Bilbo*); *mota* en minúsculas y sin espacios.
- *potentzia* y *prezioa* como números decimales (sin unidades); *24ordu* como True/False; *lat* y *lon* como decimales.

3. Analizar con consultas **(3 puntos)**

- **A)** Número de cargadores y potencia media por *mota*.
- **B)** Precio medio por provincia.
- **C)** Los 10 cargadores más potentes.
- **D)** En cada provincia: cuántos abren 24 h, cuántos no y el % de 24 h.
- **E)** Datos de mapa: *lon, lat, mota*.

4. Presentar **(3 puntos)**

- Guarda **cada tabla en un CSV**, ábrelo en Excel y haz los gráficos indicados:
- A → columnas · B → columnas · C → barras (orden inverso) · D → columnas apiladas (solo las 2 columnas de conteo) · E → dispersión con 2 colores (una columna auxiliar de latitud por *mota*, con `NOD()`).

Ejercicio 2 · Ikastetxeak (colegios)

Datos de colegios con números que a veces vienen como texto, como «n/a» o vacíos.

Archivo: `ikastetxeak.json` · Solución: `ebazpena2.py`

Campo	Ejemplo (tal como viene, sucio)
izena, udalerria, probintzia	"Mendi 37", "Laudio", "Araba"
mota	"Publikoa", "PUBLIKOA", "itunpekoa "
ikasleak	108, "350", "n/a", null
irakasleak	6, "6", ""
eredua	"D", "b", "a"
urtea	"1967", 1967, ""

1. Importar **(1 punto)**

- Carga el JSON en MongoDB (colección `gordinak`).

2. Limpiar **(3 puntos)**

- Lee `gordinak` y guarda en `garbiak` solo los documentos buenos, ya limpios:
- Descarta los colegios con *ikasleak* no válido o sin *irakasleak*.
- *ikasleak*, *irakasleak* y *urtea* como enteros (si *urtea* no es válido, deja **None** pero conserva el colegio).
- *mota* en minúsculas y *eredua* en mayúsculas.
- Campos nuevos: *ratioa* = alumnos / profesores (1 decimal) y *hamarkada* (1987 → 1980).

3. Analizar con consultas **(3 puntos)**

- **A)** Ratio medio alumnos/profesor por *mota*.
- **B)** Colegios de cada *eredua* (A, B, D) en cada provincia.
- **C)** Los 10 colegios con más alumnos.
- **D)** Colegios abiertos por década (ignora los sin año).
- **E)** Alumnos totales por provincia.

4. Presentar **(3 puntos)**

- Guarda **cada tabla en un CSV**, ábrelo en Excel y haz los gráficos indicados:
- A → columnas · B → columnas apiladas · C → barras · D → líneas o columnas · E → sectores (tarta).

Ejercicio 3 · Mendi-ibilbideak (rutas de montaña, con una lista dentro)

Rutas con distancias y duraciones en texto, y una **lista** de estaciones recomendadas en cada ruta.

Archivo: `ibilbideak.json` · Solución: `ebazpena3.py`

Campo	Ejemplo (tal como viene, sucio)
izena, herria, probintzia	"Gorbea bira 4", "Eibar", "Gipuzkoa"
distantzia	"12,5 km"
iraupena	"3h 30min", "3h30min", "2h", "45min"
desnibela	"850 m"
zailtasuna	"Erraza", "erraza ", "ZAILA"
urtaroak	["uda", "UDAZKENA"] o **[]**

1. Importar (1 punto)

- Carga el JSON en MongoDB (colección `gordinak`).

2. Limpiar (3 puntos)

- Lee `gordinak` y guarda en `garbiak` solo los documentos buenos, ya limpios:
- *distantzia* (decimal) y *desnibela* (entero) sin unidades.
- *iraupena* convertida a **minutos** (campo *minutuak*): "3h 30min" → 210.
- *zailtasuna* en minúsculas y sin espacios; cada estación de *urtaroak* en minúsculas.
- Si *urtaroak* está vacía, pon `["urte osoa"]`.

3. Analizar con consultas (3 puntos)

- **A)** Rutas de cada dificultad en cada provincia.
- **B)** Distancia media por dificultad.
- **C)** Rutas por estación (usa `$unwind`: abre la lista en un documento por estación).
- **D)** Las 10 rutas más largas en minutos.
- **E)** Distancia y desnivel de cada ruta.

4. Presentar (3 puntos)

- Guarda **cada tabla en un CSV**, ábrelo en Excel y haz los gráficos indicados:
- A → columnas apiladas · B → columnas · C → columnas · D → barras · E → dispersión (X = distancia, Y = desnivel).

Ejercicio 4 · Salmentak (ventas de una tienda)

Ventas con fechas *dd/mm/aaaa*, precios en formato europeo (con punto de miles) y cantidades erróneas.

Archivo: `salmentak.json` · Solución: `ebazpena4.py`

Campo	Ejemplo (tal como viene, sucio)
data	"13/07/2024", "05/03/**2023**", ""
produktua	"Telefonoa"
kategoria	"Arropa", "ARROPA", "arropa"
kopurua	"3", "0", "-1"
prezioa	"12,50 €", "1.250,50 €"
herria	"Bilbo"
ordainketa	"Txartela", "TXARTELA", "eskudirua"

1. Importar **(1 punto)**

- Carga el JSON en MongoDB (colección `gordinak`).

2. Limpiar **(3 puntos)**

- Lee `gordinak` y guarda en `garbiak` solo los documentos buenos, ya limpios:
- Quédate solo con las ventas de **2024** y con *kopurua* mayor que 0 (descarta fecha vacía, otros años, 0 y negativos).
- *prezioa* como decimal (`"1.250,50 €"` → 1250.5) y *kopurua* como entero.
- *kategoria* y *ordainketa* con la primera letra en mayúscula (`capitalize()`).
- Campos nuevos: *hilabetea* (número de mes) y *guztira* = *kopurua* × *prezioa*.

3. Analizar con consultas **(3 puntos)**

- **A)** Ventas totales de cada mes (los 12 meses, con 0 si no hay).
- **B)** Ventas por categoría.
- **C)** Nº de ventas por forma de pago en cada categoría.
- **D)** Los 5 productos que más ingresan.
- **E)** Importe medio de una venta por pueblo.

4. Presentar **(3 puntos)**

- Guarda **cada tabla en un CSV**, ábrelo en Excel y haz los gráficos indicados:
- A → líneas · B → sectores o columnas · C → columnas apiladas · D → barras · E → columnas.

Plantilla para empezar

Está también como archivo `plantilla_examen.py`. Cambia los `ZUZENDU` por tus campos.

Esquema de cada consulta

```
A = [["titulo1", "titulo2"]]                       # primera fila = títulos
for d in db.garbiak.aggregate([
    {"$match": {...}},                              # (opcional) filtrar
    {"$group": {"_id": "$campo", "kopurua": {"$sum": 1}}},    # agrupar y contar ($sum:1) / sumar ($sum:"$x") /
    {"$sort": {"kopurua": -1}},                     # (opcional) ordenar
    {"$limit": 10},                                 # (opcional) top N
]):
    A.append([d["_id"], d["kopurua"]])
csv_idatzi("taula_A.csv", A)
```

Si agrupas por dos campos, el resultado se lee así:

```
kopuruak = {}
for d in db.garbiak.aggregate([
    {"$group": {"_id": {"a": "$campo1", "b": "$campo2"}, "kopurua": {"$sum": 1}}},
]):
    kopuruak[(d["_id"]["a"], d["_id"]["b"])] = d["kopurua"]
# y luego, para cada fila de la tabla:   kopuruak.get((a, b), 0)
```

Antes de entregar: 1) ¿el `print` del número de documentos limpios tiene sentido? 2) ¿los CSV abren en columnas en Excel? 3) ¿cada gráfico tiene título y ejes con nombre? 4) ¿has guardado el Excel como .xlsx (los CSV no guardan gráficos)?